7-Layer Advanced RAG — Banking Assistant

Complete Technical Documentation

Author: Optimum AI Lab **Date:** March 11, 2026 **Version:** 1.0.0

Table of Contents

1. System Overview
2. Architecture
3. Project Structure
4. Configuration Reference
5. Core RAG Engine — 7-Layer Pipeline
6. FAQ Smart Router — 3-Tier Query Routing
7. Multi-Strategy Document Extraction
8. 9 RAG Innovations
9. Product Orchestrator — Layer 0
10. AI Governance Engine — Four-Check System
11. Persistence Layer — MongoDB + FAISS
12. Turbo Mode
13. Admin Document Management
14. Prompt Management Layer
15. User Interface — Streamlit Application
16. Recent Bug Fixes and Improvements
17. Local Setup Guide

18. API and Model Dependencies

---

1. System Overview

The **7-Layer Advanced RAG Banking Assistant** is a production-grade Retrieval-Augmented Generation system designed for the banking domain. It processes customer queries through a multi-layered pipeline that includes semantic caching, query transformation, intelligent retrieval, corrective RAG, re-ranking, agentic reasoning, and response validation — all governed by a four-check AI governance engine aligned with CBUAE regulations.

The system supports two operational modes:

- **Full Pipeline Mode:** All 7 layers + 9 innovations + governance (10–20 seconds, maximum accuracy)
- **Turbo Mode:** FAQ match + vector search + 1 LLM call (under 3 seconds, optimized for FAQ-heavy workloads)

The application is built with **Streamlit** for the frontend, **MongoDB** for document and metadata persistence, **FAISS** for vector storage, and **OpenAI-compatible LLMs** for all generative tasks.

---

2. Architecture

High-Level Data Flow

```
User Query
    │
    ▼
┌──────────────────────────────────────────────────────┐
│  Layer 0: Product Orchestrator                       │
│  (Classification → Intent → Risk → Routing)          │
└──────────────────────────┬───────────────────────────┘
                           │
           ┌───────────────┴────────────────┐
           │  Turbo Mode?                   │
           │  YES           NO              │
           ▼                ▼               │
   ┌──────────┐   ┌──────────────┐         │
   │ FAQ Match │   │ Layer 1-7    │         │
   │ + Vector  │   │ Full Pipeline│         │
   │ + 1 LLM   │   │ + Innovations│         │
   └──────────┘   └──────────────┘         │
           │              │                 │
           └──────┬───────┘                 │
                  ▼                         │
   ┌──────────────────────┐                 │
   │ Governance Engine    │                 │
   │ (4-Check Validation) │                 │
   └──────────┬───────────┘                 │
              ▼                             │
        Final Response                      │
└──────────────────────────────────────────────────────┘
```

Document Ingestion Flow

```
PDF Upload
    │
    ▼
┌──────────────────────────────────────────────────────┐
│  Extraction Orchestrator                             │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌────────┐│
│  │ Vision   │ │ Table    │ │ Graph    │ │Enrich │
│  │Extractor│ │Extractor│ │ Builder │ │Engine │
│  └────┬─────┘ └────┬─────┘ └────┬─────┘ └───┬────┘│
│       └────────────┴────────────┴────────────┘     │
│                        │                            │
│              Fusion & Dedup                          │
└──────────────────────┬──────────────────────────────┘
                       │
          ┌────────────┴────────────┐
          ▼                         ▼
   ┌──────────────┐          ┌──────────────┐
   │ FAQ Pairs    │          │ Enriched     │
   │ Extraction   │          │ Chunks       │
   │ (Q&A Pairs)  │          │ (Indexed)    │
   └──────┬───────┘          └──────┬───────┘
          │                         │
          ▼                         ▼
   ┌──────────────┐          ┌──────────────┐
   │ MongoDB      │          │ MongoDB      │
   │ faq_pairs    │          │ chunks       │
   └──────────────┘         │ + FAISS      │
                            └──────────────┘
```

3. Project Structure

```
banking-rag-app/
├── app.py                          # Main Streamlit application (909 lines)
├── requirements.txt                # Python dependencies
├── LOCAL_SETUP.md                  # Local deployment guide
├── .env.example                    # Environment variable template
│
├── config/
│   └── settings.py                 # Centralized configuration (213 lines,
40+ toggles)
│
├── rag_engine/
│   ├── seven_layer_rag.py          # Core 7-layer RAG engine (2228 lines)
│   ├── faq_exact_match.py          # FAQ Smart Router — 3-tier routing (599
lines)
│   ├── document_chunker.py         # Semantic document chunking (125 lines)
│   ├── product_orchestrator.py     # Product classification & routing (276
lines)
│   │
│   ├── extractors/
│   │   ├── extraction_orchestrator.py  # Multi-strategy extraction pipeline
(1046 lines)
│   │   ├── vision_extractor.py         # LLM vision-based extraction (375
lines)
│   │   ├── table_extractor.py          # Structured table extraction (635
lines)
│   │   ├── knowledge_graph_builder.py  # Entity-relationship graph (578
lines)
│   │   └── contextual_enrichment.py    # Contextual prefix enrichment (420
lines)
│   │
│   └── innovations/
│       ├── hybrid_search.py            # BM25 + Vector with RRF (132 lines)
│       ├── contextual_retrieval.py     # Anthropic-style context prefixing
(92 lines)
│       ├── graph_rag.py                # Knowledge graph RAG (132 lines)
│       ├── adaptive_rag.py             # Dynamic strategy selection (73
lines)
│       ├── query_decomposition.py      # Multi-part query breakdown (88
lines)
│       ├── self_rag.py                 # Self-reflective retrieval (122
lines)
│       ├── speculative_rag.py          # Parallel draft generation (178
```

```
lines)
|       ├── raptor_indexing.py          # Hierarchical tree summaries (149
lines)
|       └── ragas_evaluation.py         # RAGAS quality metrics (97 lines)
|
├── persistence/
|   ├── persistence_manager.py      # Unified persistence orchestrator (415
lines)
|   ├── mongo_store.py              # MongoDB document/chunk/FAQ store (388
lines)
|   └── faiss_store.py              # FAISS vector index store (263 lines)
|
├── governance/
|   └── governance_engine.py        # 4-check governance system (308 lines)
|
├── prompts/
|   └── prompt_manager.py           # Centralized prompt templates (318
lines)
|
├── feedback/
|   └── feedback_service.py         # User feedback collection (88 lines)
|
├── ui/
|   └── detailed_layer_display.py   # Layer-by-layer pipeline visualization
(177 lines)
|
└── data/
    ├── faiss_index/                # FAISS index persistence (auto-
generated)
    ├── feedback/                   # User feedback JSON files
    ├── audit/                      # Governance audit trail
    ├── page_images/                # Extracted page images for vision
    └── temp/                       # Temporary processing files
```

Total Source Code: Approximately **7,800 lines** of Python across 25 source files.

---

4. Configuration Reference

All configuration is centralized in `config/settings.py` using Python dataclasses. Every feature is individually toggleable.

Application Configuration (`AppConfig`)

Parameter	Type	Default	Description
`app_title`	str	“7-Layer Advanced RAG — Banking Assistant”	Application title
`app_icon`	str	“🏦”	Favicon/icon
`version`	str	“1.0.0”	Application version

Authentication (`AuthConfig`)

Parameter	Type	Default	Description
`admin_username`	str	“admin”	Admin login username
`admin_password`	str	“admin88$”	Admin login password
`session_timeout_minutes`	int	60	Session expiry time

LLM Configuration (`LLMConfig`)

Parameter	Type	Default	Description
`model_name`	str	“gpt-4.1-mini”	Primary LLM model
`embedding_model`	str	“text-embedding-3-small”	OpenAI embedding model
`embedding_dimension`	int	1536	Embedding vector dimension
`temperature`	float	0.1	LLM temperature
`max_tokens`	int	2048	Max response tokens
`api_key`	str	env `OPENAI_API_KEY`	OpenAI API key

RAG Layer Configuration (`RAGLayerConfig`)

Layer 1: Semantic Caching

Parameter	Type	Default	Description
`cache_enabled`	bool	True	Enable/disable semantic cache
`cache_similarity_threshold`	float	0.92	Minimum similarity for cache hit
`cache_ttl_seconds`	int	3600	Cache entry time-to-live
`cache_max_entries`	int	500	Maximum cached entries

Layer 2: HyDE (Query Transformation)

Parameter	Type	Default	Description
`hyde_enabled`	bool	True	Enable Hypothetical Document Embeddings
`hyde_min_query_length`	int	10	Minimum query length to trigger HyDE

Layer 3: Semantic Chunking & Retrieval

Parameter	Type	Default	Description
`chunk_size_min`	int	200	Minimum chunk size (characters)
`chunk_size_max`	int	800	Maximum chunk size (characters)
`chunk_overlap`	int	50	Overlap between chunks
`retrieval_top_k`	int	20	Number of chunks to retrieve

Layer 4: Corrective RAG (CRAG)

Parameter	Type	Default	Description
`crag_enabled`	bool	True	Enable corrective retrieval
`crag_quality_threshold_correct`	float	0.7	Threshold for “correct” grading
`crag_quality_threshold_ambiguous`	float	0.4	Threshold for “ambiguous” grading
`crag_web_fallback_enabled`	bool	True	Enable web search fallback
`crag_trusted_urls`	list	[mashreqbank.com, centralbank.ae]	Trusted web sources

Layer 5: LLM Re-Ranking

Parameter	Type	Default	Description
`rerank_enabled`	bool	True	Enable LLM-based re-ranking
`rerank_top_k`	int	5	Final top-K after re-ranking
`rerank_diversity_enabled`	bool	True	Enable diversity in re-ranking

Layer 6: Agentic RAG (ReAct)

Parameter	Type	Default	Description
`agentic_enabled`	bool	True	Enable agentic reasoning
`agentic_max_iterations`	int	5	Max ReAct iterations
`agentic_complexity_threshold`	float	0.6	Complexity threshold to activate

Layer 7: Response Validation

Parameter	Type	Default	Description
`validation_enabled`	bool	True	Enable response validation
`validation_confidence_high`	float	0.70	Confidence threshold for delivery
`validation_confidence_medium`	float	0.30	Confidence threshold for warning
`hallucination_check_enabled`	bool	True	Enable hallucination detection

FAQ Smart Router

Parameter	Type	Default	Description
`faq_exact_match_enabled`	bool	True	Master switch for FAQ routing
`faq_exact_threshold`	float	0.85	Tier 1 (EXACT) similarity threshold
`faq_fuzzy_threshold`	float	0.60	Tier 2 (FUZZY) similarity threshold
`faq_fuzzy_model`	str	“gpt-4.1-nano”	Lightweight model for fuzzy adaptation

9 RAG Innovations

Parameter	Type	Default	Description
`hybrid_search_enabled`	bool	True	BM25 + Vector with RRF
`hybrid_search_bm25_weight`	float	0.3	BM25 weight in fusion
`hybrid_search_vector_weight`	float	0.7	Vector weight in fusion
`contextual_retrieval_enabled`	bool	True	Anthropic-style context prefixing
`ragas_evaluation_enabled`	bool	True	RAGAS quality metrics
`graph_rag_enabled`	bool	True	Knowledge graph reasoning
`adaptive_rag_enabled`	bool	True	Dynamic strategy selection
`query_decomposition_enabled`	bool	True	Multi-part query breakdown
`self_rag_enabled`	bool	True	Self-reflective retrieval
`speculative_rag_enabled`	bool	True	Parallel draft generation
`raptor_enabled`	bool	True	Hierarchical tree summaries

Document Extraction

Parameter	Type	Default	Description
`extraction_orchestrator_enabled`	bool	True	Master switch for advanced extraction
`extraction_config.fast_mode`	bool	True	Fast mode (skip vision + graph)
`extraction_config.vision_enabled`	bool	True	LLM vision extraction
`extraction_config.table_enabled`	bool	True	Structured table extraction
`extraction_config.graph_enabled`	bool	True	Knowledge graph construction
`extraction_config.contextual_enabled`	bool	True	Contextual enrichment
`extraction_config.vision_max_pages`	int	3	Max pages for vision analysis
`extraction_config.graph_max_chunks`	int	5	Max chunks for graph construction

Governance Engine

Parameter	Type	Default	Description
`governance_enabled`	bool	True	Master switch for governance
`governance_check1_hallucination`	bool	True	Factual correctness check
`governance_check2_bias`	bool	True	Bias and toxicity check
`governance_check3_pii`	bool	True	PII redaction check
`governance_check4_compliance`	bool	True	Regulatory compliance check
`governance_audit_trail_enabled`	bool	True	14-field audit records
`governance_max_retries`	int	3	Max retries on hallucination failure

MongoDB Configuration (`MongoDBConfig`)

Parameter	Type	Default	Description
`enabled`	bool	True	Enable MongoDB persistence
`connection_string`	str	env `MONGODB_URI` or `mongodb://localhost:27017`	MongoDB connection URI
`database_name`	str	"banking_rag"	Database name
`embedding_dimension`	int	1536	Vector dimension
`batch_size`	int	100	Bulk insert batch size
`ingestion_auto_detect_product`	bool	True	Auto-classify documents
`ingestion_require_approval`	bool	True	Require admin approval
`ingestion_pii_scan_enabled`	bool	True	PII scan before indexing

---

5. Core RAG Engine — 7-Layer Pipeline

File: `rag_engine/seven_layer_rag.py` (2,228 lines)

The `SevenLayerRAG` class is the central orchestrator. Each query passes through up to 7 layers sequentially, with early exit when confidence is sufficient.

Layer 0: Product Orchestration

- **Purpose:** Classify the query by banking product, analyze intent, assess risk, and route to the appropriate vector store.
- **Components:** Product classification (keyword + LLM), intent analysis (inquiry/complaint/transaction/etc.), risk scoring (low/medium/high/critical), confidence assessment.
- **LLM Calls:** 0–1 (only if keyword confidence < 0.75).

Layer 1: Semantic Caching

- **Purpose:** Check if a semantically similar query has been answered recently.
- **Mechanism:** Embeds the query and compares against cached query embeddings using cosine similarity. If similarity >= 0.92, returns the cached response.
- **LLM Calls:** 0.
- **Latency:** ~10ms.

Layer 2: HyDE (Hypothetical Document Embeddings)

- **Purpose:** Transform the user query into a hypothetical ideal answer paragraph, then use that paragraph's embedding for retrieval (improves recall for short/vague queries).
- **Mechanism:** One LLM call generates a hypothetical answer. The embedding of this hypothetical answer is used for vector search instead of the raw query embedding.
- **LLM Calls:** 1.

Layer 3: Semantic Retrieval

- **Purpose:** Retrieve the top-K most relevant document chunks using vector similarity search.
- **Mechanism:** Cosine similarity between the query (or HyDE) embedding and all chunk embeddings. Returns top 20 chunks.
- **Enhancements:** If hybrid search is enabled, combines BM25 lexical search with vector search using Reciprocal Rank Fusion (RRF).

- **LLM Calls:** 0.

Layer 4: Corrective RAG (CRAG)

- **Purpose:** Grade each retrieved chunk for relevance and filter out irrelevant ones. If all chunks are irrelevant, trigger web search fallback.
- **Mechanism:** Each chunk is graded by the LLM as “correct”, “ambiguous”, or “incorrect”. Incorrect chunks are discarded. If no correct chunks remain, web search is triggered.
- **LLM Calls:** 1 per chunk (up to 20).

Layer 5: LLM Re-Ranking

- **Purpose:** Re-rank the surviving chunks by fine-grained relevance scoring and select the final top-K.
- **Mechanism:** Each chunk receives a 0–10 relevance score from the LLM. Chunks are sorted by score and the top 5 are selected.
- **LLM Calls:** 1 per chunk (up to 10).

Layer 6: Agentic RAG (ReAct)

- **Purpose:** For complex queries, use iterative reasoning with tool use (Thought → Action → Observation loop).
- **Mechanism:** The LLM reasons about the query, decides whether to search for more information, analyze documents, or calculate values. Runs up to 5 iterations.
- **LLM Calls:** 1–5 (iterative).
- **Activation:** Only for queries above the complexity threshold (0.6).

Layer 7: Response Generation & Validation

- **Purpose:** Generate the final answer from the top-K chunks and validate it for quality.
- **Mechanism:** One LLM call generates the answer. A separate validation call checks confidence, completeness, and factual grounding.

- **LLM Calls:** 1–2.
- **Output:** `RAGResponse` with answer, confidence score, sources, layer results, and validation status.

Key Data Structures

```
@dataclass
class ChunkMetadata:
    text: str
    source: str
    page: int = 0
    section: str = ""
    chunk_id: str = ""

@dataclass
class RAGResponse:
    answer: str
    confidence: float
    sources: List[Dict]
    layer_results: List[LayerResult]
    total_duration_ms: float
    pipeline_stopped_at: int
    validation_status: str
    orchestration_result: Optional[OrchestrationResult]

@dataclass
class LayerResult:
    layer_number: int
    layer_name: str
    status: str
    duration_ms: float
    details: Dict[str, Any]
```

---

6. FAQ Smart Router — 3-Tier Query Routing

File: `rag_engine/faq_exact_match.py` (599 lines)

The FAQ Smart Router dramatically reduces latency for FAQ-type queries by bypassing the full 7-layer pipeline.

How It Works

During Document Upload:

1. The `extract_faq_pairs()` method scans the document text for Q&A patterns using regex (e.g., `Q1.`, `Q25.`, `Q100.`).
2. Each detected Q&A block is split into a question and answer using the `_split_question_answer()` method.
3. The answer text is cleaned by `_extract_direct_answer()` (minimal whitespace cleanup, no truncation).
4. Each question is embedded using the sentence-transformers model (`all-MiniLM-L6-v2`).
5. The resulting `FAQPair` objects are stored in memory and persisted to MongoDB.

During Query:

1. The user's query is embedded.
2. Cosine similarity is computed against ALL stored FAQ question embeddings in a single vectorized operation.
3. The highest similarity score determines the routing tier:

Tier	Similarity	Action	LLM Calls	Latency
1 (EXACT)	>= 0.85	Return stored answer directly	0	50ms
2 (FUZZY)	0.60–0.85	Adapt answer with 1 lightweight LLM call	1 (gpt-4.1-nano)	1–2s
3 (NOVEL)	< 0.60	Fall through to full 7-layer pipeline	5–20+	10–20s

Multi-Part Question Splitting (`_split_question_answer`)

The method handles complex FAQ questions that contain multiple `?` marks (e.g., "How do I get complimentary access to Fitness First? Also, is the benefit applicable for Supplementary Cardholders as well?").

Algorithm:

1. Find all `?` positions in the QA block text.
2. Skip `?` marks inside URLs (detected by checking for `.html?`, `.php?`, `http://`, `.com?`, etc.).
3. For each remaining `?`, check if the text after it starts with an **answer-like word** (e.g., “You”, “The”, “To”, “Yes”, “No”, “This”) vs. a **question continuation word** (e.g., “Also”, “What”, “How”, “Is”, “Can”).
4. Select the **last `?`** that is followed by answer-like text as the split point.
5. If no good split is found, fall back to the first `?`.

Answer Extraction (`_extract_direct_answer`)

The method performs minimal cleanup on the raw answer text:

- Strips leading/trailing whitespace.
- Collapses multiple newlines into single newlines.
- Collapses multiple spaces into single spaces.
- **No truncation, no character limit, no splitting on capital letters.**

Fuzzy Adaptation (`adapt_answer_for_fuzzy_match`)

For Tier 2 (FUZZY) matches, the LLM is called with explicit instructions to **reproduce the FULL and COMPLETE answer text** from the FAQ document. The prompt explicitly prohibits summarizing, shortening, or paraphrasing. Max tokens: 1500.

---

7. Multi-Strategy Document Extraction

File: `rag_engine/extractors/extraction_orchestrator.py` (1,046 lines)

The `ExtractionOrchestrator` processes uploaded documents through multiple extraction strategies and fuses the results into enriched chunks.

Extraction Strategies

Strategy	File	Description	When Used
Vision	`vision_extractor.py`	LLM vision API processes page images to extract text, tables, and formulas	Full mode, or scanned PDF fallback
Table	`table_extractor.py`	pdfplumber/Docling structured table extraction with coordinate mapping	Always (if tables detected)
Knowledge Graph	`knowledge_graph_builder.py`	Extracts entities and relationships for cross-document reasoning	Full mode only
Contextual Enrichment	`contextual_enrichment.py`	Adds contextual prefixes to chunks, linearizes formulas, enriches metadata	Always

Processing Modes

Fast Mode (`fast_mode: True` , default):

- Uses only: Table + Enrichment strategies.
- Skips: Vision + Knowledge Graph.
- Processing time: ~10–30 seconds.
- **Exception:** If the PDF is detected as scanned (no text layer), Vision API is automatically activated as a fallback even in Fast Mode.

Full Mode (`fast_mode: False`):

- Uses all 4 strategies.
- Processing time: ~60–120 seconds.
- Best for complex documents with images, charts, and cross-references.

Scanned PDF Vision API Fallback

When a PDF has no extractable text layer (scanned/image-only), the orchestrator automatically detects this condition and activates the Vision API extractor regardless of the Fast Mode setting. This ensures that scanned documents are properly processed.

Detection logic: After PyMuPDF text extraction, if both `text_chunks` and `full_text` are empty, the fallback triggers.

Fusion and Deduplication

After all strategies complete, the orchestrator:

1. Collects chunks from all strategies.
2. Deduplicates by text similarity (Jaccard similarity > 0.8).
3. Merges metadata from different strategies.
4. Assigns unique chunk IDs.
5. Returns enriched chunks ready for indexing.

Human-in-the-Loop Fallback

If all extraction strategies fail for a document, the orchestrator emits a critical failure event and flags the document for human review.

---

8. 9 RAG Innovations

All innovations are individually toggleable via `config/settings.py`.

Horizon 1: High Feasibility

1. Hybrid Search (`innovations/hybrid_search.py`)

Combines BM25 lexical search with vector semantic search using Reciprocal Rank Fusion (RRF). Default weights: BM25 = 0.3, Vector = 0.7. Includes `remove_by_source()` for document deletion support.

2. Contextual Retrieval (`innovations/contextual_retrieval.py`)

Implements Anthropic's context-prefix technique. Each chunk is enriched with a contextual prefix that describes where it fits within the document, improving retrieval accuracy for ambiguous queries.

3. RAGAS Evaluation (`innovations/ragas_evaluation.py`)

Computes RAGAS quality metrics per query: faithfulness, answer relevancy, context precision, and context recall. Sample rate is configurable (default: 100%).

Horizon 2: Medium Feasibility

4. Knowledge Graph RAG (`innovations/graph_rag.py`)

Builds a knowledge graph from extracted entities and relationships. Enables cross-document reasoning by traversing the graph (configurable max hops: 2).

5. Adaptive RAG (`innovations/adaptive_rag.py`)

Dynamically selects the retrieval strategy based on query complexity. Simple queries use direct retrieval; complex queries activate additional layers.

6. Query Decomposition (`innovations/query_decomposition.py`)

Breaks multi-part queries into sub-queries (max 5), processes each independently, and synthesizes the results into a unified answer.

Horizon 3: Research-Grade

7. Self-RAG (`innovations/self_rag.py`)

Self-reflective retrieval with correction loops. After generating an initial answer, the system evaluates whether the answer is supported by the retrieved context. If not, it re-retrieves and regenerates (max 3 iterations).

8. Speculative RAG (`innovations/speculative_rag.py`)

Generates multiple draft answers in parallel (default: 3 drafts) using different context subsets, then verifies and selects the best draft.

9. RAPTOR Indexing (`innovations/raptor_indexing.py`)

Builds a hierarchical tree of document summaries at multiple levels of abstraction (max 3 levels). Enables retrieval at different granularity levels.

---

9. Product Orchestrator — Layer 0

File: `rag_engine/product_orchestrator.py` (276 lines)

The Product Orchestrator classifies incoming queries into banking product categories and routes them to the appropriate vector stores.

Product Catalog

Product	Collection	Keywords
Accounts	`accounts`	account, savings, current, deposit, balance, statement
Cards	`cards`	card, credit card, debit card, visa, mastercard, chargeback
Loans	`loans`	loan, mortgage, personal loan, interest rate, EMI
Transfers	`transfers`	transfer, wire, remittance, SWIFT, IBAN
Insurance	`insurance`	insurance, policy, claim, premium, coverage
Investments	`investments`	investment, mutual fund, stocks, portfolio

Orchestration Pipeline

1. **Product Classification:** Keyword matching first (fast). If confidence < 0.75, LLM classification is used.
2. **Intent Analysis:** Classifies the query intent (inquiry, complaint, transaction, etc.).
3. **Risk Assessment:** Scores the query risk (low/medium/high/critical) based on financial impact and regulatory sensitivity.
4. **Routing:** Directs the query to the appropriate product-specific vector store.

Output Data Structures

- `IntentResult`: primary intent, confidence, secondary intents, human handoff flag.
- `RiskResult`: risk score (0–1), risk level, risk factors, recommended action.
- `ConfidenceResult`: product confidence, keyword match score, semantic alignment score.
- `OrchestrationResult`: combined product, intent, risk, confidence, and routing decision.

---

10. AI Governance Engine — Four-Check System

File: `governance/governance_engine.py` (308 lines)

The Governance Engine validates every response through four sequential checks aligned with CBUAE (Central Bank of UAE) regulations.

Check 1: Hallucination & Factual Correctness

- Verifies that the response is grounded in the retrieved context.
- Detects fabricated facts, unsupported claims, and invented numbers.
- If hallucination is detected, the response is regenerated (up to 3 retries).

Check 2: Bias, Toxicity & Fairness

- Scans for discriminatory language, gender bias, racial bias, and toxic content.
- Ensures responses are neutral and professional.

Check 3: PII & Sensitive Data Redaction

- Pattern-based detection of PII: email addresses, phone numbers, Emirates IDs, credit card numbers, IBANs, passport numbers.
- Automatically redacts detected PII with `[REDACTED]` markers.

Check 4: Regulatory & Compliance Validation

- Validates responses against banking regulatory requirements.
- Checks for unauthorized financial advice, missing disclaimers, and compliance violations.

Governance Output

```python
@dataclass
class GovernanceResult:
    overall_status: str  # "approved", "warning", "blocked", "escalated"
    final_response: str
    original_response: str
    checks: List[GovernanceCheckResult]
    modifications_made: List[str]
    escalated_to_human: bool
    retry_count: int
    total_duration_ms: float
```

Audit Trail

When enabled, every query generates a 14-field audit record stored in `data/audit/audit_trail.json`:

- Timestamp, query, response, confidence, product, intent, risk level, governance status, checks passed/failed, modifications, duration, model used, sources, session ID.

---

11. Persistence Layer — MongoDB + FAISS

MongoDB Store (`persistence/mongo_store.py`, 388 lines)

Collections:

Collection	Purpose	Key Fields
`documents`	Document metadata	filename, file_hash, page_count, file_size, strategies, status
`chunks`	Document chunks	chunk_id, text, source, page, section, chunk_type
`tables`	Extracted tables	source, page, table_data, headers
`graph_entities`	Knowledge graph entities	entity_id, name, type, source
`graph_relationships`	Knowledge graph edges	source_entity, target_entity, relationship
`faq_pairs`	FAQ question-answer pairs	faq_id, question, answer, question_number, source, embedding

Key Methods:

- `store_document()` / `document_exists()` / `document_hash_exists()` — Document deduplication by content hash.
- `store_chunks()` / `get_all_chunks()` / `get_chunks_by_source()` — Chunk CRUD.
- `store_faq_pairs()` / `get_all_faq_pairs()` / `delete_faq_pairs_by_source()` — FAQ pair persistence.
- `delete_document()` — Cascading delete (document + chunks + tables + entities + relationships + FAQ pairs).

FAISS Store (`persistence/faiss_store.py` , 263 lines)

- Uses `IndexFlatIP` (Inner Product with normalized vectors = cosine similarity).
- Dimension: 384 (for `all-MiniLM-L6-v2` local model).
- Persists index and metadata to disk (`data/faiss_index/`).
- Metadata mapping: `faiss_id → {chunk_id, source, text_preview, section}` .

- `remove_document()` rebuilds the entire index after filtering out the document's vectors.
- Consistency check on load: if vector count does not match metadata count, the index is reset.

Persistence Manager (`persistence/persistence_manager.py`, 415 lines)

Orchestrates MongoDB + FAISS operations:

- `load_into_indexer()` — On startup, loads chunks from MongoDB and vectors from FAISS into the in-memory `DocumentIndexer`. **Includes FAISS/chunk sync logic** that handles mismatches by matching vectors to chunks by `chunk_id` and re-embedding any unmatched chunks.
- `store_document()` / `store_chunks()` — Stores to both MongoDB and FAISS.
- `delete_document()` — Deletes from both MongoDB and FAISS.
- `document_exists()` — Deduplication check.

FAISS/Chunk Sync on Startup

When the FAISS vector count does not match the MongoDB chunk count (e.g., after a document was deleted from MongoDB but FAISS was not cleaned up), the persistence manager:

1. Builds a `chunk_id` mapping from MongoDB chunks.
2. Matches each FAISS vector to its corresponding chunk via metadata `chunk_id`.
3. Re-embeds any unmatched chunks using the embedding model.
4. Rebuilds the FAISS index to match the synced state.
5. Saves the rebuilt index to disk.

This prevents the "list index out of range" error in Turbo Mode vector search.

---

12. Turbo Mode

Location: `rag_engine/seven_layer_rag.py`, method `_turbo_pipeline()` (lines 1983–2150)

Turbo Mode is an ultra-fast query pipeline that bypasses all heavy layers for maximum speed.

Pipeline

```
Query → FAQ Exact Match → Vector Search → 1 LLM Call → Answer
```

Steps

1. **FAQ Exact Match** (0 LLM calls, ~50ms): Checks if the query matches a stored FAQ. If EXACT match (>= 0.85), returns immediately. If FUZZY match (0.60–0.85), adapts with 1 lightweight LLM call.
2. **Simple Vector Search** (0 LLM calls, ~100ms): Direct cosine similarity search against all chunk embeddings. No HyDE, no hybrid search.
3. **Single LLM Answer Generation** (1 LLM call): Generates the answer from the top-5 retrieved chunks using a prompt that instructs the LLM to reproduce FULL and COMPLETE FAQ answers.

Performance

Metric	Turbo Mode	Full Pipeline
FAQ Exact Hit	~50ms, 0 LLM calls	~10–20s, 5–20+ LLM calls
FAQ Fuzzy Hit	~1–2s, 1 LLM call	~10–20s, 5–20+ LLM calls
Novel Query	~3–5s, 1 LLM call	~10–20s, 5–20+ LLM calls

Activation

Toggle in the sidebar under “Layer Configuration” → “TURBO MODE” toggle.

13. Admin Document Management

Location: `app.py`, Admin Panel tab (lines 739–850)

Features

The Admin Panel includes a **Document Management** section that allows administrators to:

1. **View all uploaded documents** in a detailed table showing:
    - Filename
    - Number of chunks
    - Number of tables extracted
    - Number of entities extracted
    - Page count
    - File size (KB/MB)
    - Product category
    - Upload timestamp
    - Extraction strategies used
    - Processing status
2. **Delete documents** with a confirmation dialog. Deletion removes the document from all 5 stores:
    - **MongoDB** (document record, chunks, tables, entities, relationships, FAQ pairs)
    - **FAISS** (vector embeddings — index is rebuilt)
    - **In-memory DocumentIndexer** (chunks and embeddings array)
    - **FAQ Engine** (FAQ pairs for that document)
    - **Hybrid Search** (BM25 index entries for that document)
    - **Semantic Cache** (cleared after deletion to prevent stale results)

3. **Summary statistics** displayed at the top:
    - Total documents, total chunks, total FAISS vectors, total FAQ pairs.

Delete Flow

1. Admin clicks “Delete” button next to a document.
2. Confirmation dialog appears: “Are you sure you want to delete **{filename}**? This will remove it from MongoDB, FAISS, in-memory index, FAQ pairs, and hybrid search.”
3. Admin clicks “Yes, Delete” to confirm or “Cancel” to abort.
4. On confirmation, `engine.delete_document(filename)` is called, which orchestrates cleanup across all stores.
5. Page refreshes with updated counts.

Implementation

The `SevenLayerRAG.delete_document()` method (lines 720–770) orchestrates the deletion:

```
def delete_document(self, source: str) -> Dict[str, Any]:
    # 1. Remove from in-memory indexer (chunks + embeddings)
    # 2. Remove from FAQ engine
    # 3. Remove from hybrid search (BM25)
    # 4. Remove from persistence (MongoDB + FAISS)
    # 5. Clear semantic cache
    # Returns: {removed_chunks, removed_faq_pairs, ...}
```

The `SevenLayerRAG.get_all_documents()` method retrieves document metadata from MongoDB for display.

---

14. Prompt Management Layer

File: `prompts/prompt_manager.py` (318 lines)

All LLM prompts are centralized in the `PromptManager` class. No prompts are hardcoded in service code.

Prompt Templates

Method	Layer	Purpose
`hyde_generator()`	Layer 2	Generate hypothetical document for HyDE
`crag_quality_grader()`	Layer 4	Grade chunk relevance (correct/ambiguous/incorrect)
`rerank_scorer()`	Layer 5	Score chunk relevance 0–10
`agentic_react()`	Layer 6	ReAct reasoning (Thought → Action → Observation)
`response_generator()`	Layer 7	Generate final answer from context
`confidence_validator()`	Layer 7	Validate response confidence and completeness

Key Prompt Rules (Layer 7 — Response Generator)

1. Only use information from the provided context.
2. If the context does not contain the answer, say so explicitly.
3. For numerical data, quote exact figures from the context.
4. Include source references.
5. If multiple products are mentioned, clarify which product each detail applies to.
6. For procedural questions, provide step-by-step instructions.
7. Use professional banking terminology.
8. **CRITICAL — COMPLETE ANSWERS:** When the context contains FAQ entries, reproduce the FULL and COMPLETE answer text. Do NOT summarize, shorten, or paraphrase.

---

15. User Interface — Streamlit Application

File: `app.py` (909 lines)

Tabs

Tab	Access	Description
Chat	All users	Main chat interface for querying the banking assistant
Feedback	All users	View and manage user feedback entries
Admin Panel	Admin only	Document management, system stats, configuration

Sidebar

- **System Status:** Documents count, chunks count, MongoDB status, FAISS vector count.
- **Document Upload:** File uploader for PDF documents with progress tracking.
- **Layer Configuration:** Toggle individual layers, enable/disable Turbo Mode.
- **Extraction Mode:** Fast mode vs. full mode toggle.

Chat Interface

- Query input with send button.
- Response display with confidence badge (color-coded: green >= 70%, yellow >= 30%, red < 30%).
- Expandable sections:
  - **7-Layer RAG Pipeline Reasoning:** Layer-by-layer execution details.
  - **Pipeline Execution Details:** Timing, LLM calls, chunks processed.
  - **Document References:** Source documents with section and relevance score.
- Thumbs up/down feedback buttons.

Pipeline Visualization

The `ui/detailed_layer_display.py` renders each layer's execution as an expandable card showing:

- Layer name and number.

- Execution status (executed, skipped, cached, HIT).
- Duration in milliseconds.
- Detailed metrics specific to each layer.

---

16. Recent Bug Fixes and Improvements

Fix 1: FAQ Answer Truncation (`faq_exact_match.py` — `_extract_direct_answer`)

Problem: The method was aggressively truncating FAQ answers by splitting on capital letters after newlines and capping at 500 characters. Long answers (e.g., Q31 with 1,275 characters covering multiple sections a, b, c, d) were cut off.

Fix: Replaced aggressive truncation with minimal whitespace cleanup. No character limit. The full answer text is preserved exactly as it appears in the document.

Fix 2: Multi-Part Question Splitting (`faq_exact_match.py` — `_split_question_answer`)

Problem: The method split at the first `?` in the QA block. For multi-part questions like Q29 ("How do I get complimentary access to Fitness First? Also, is the benefit applicable for Supplementary Cardholders as well?"), the second part of the question was incorrectly treated as the answer.

Fix: Replaced naive first-`?` split with a smart algorithm that finds all `?` positions, skips URL query parameters (`.html?`, `.php?`, etc.), checks whether text after each `?` is answer-like or question-continuation, and selects the best split point.

Fix 3: Full Answer Reproduction in LLM Prompts

Problem: The fuzzy adaptation prompt (Tier 2) said "keep the answer concise and direct", causing the LLM to compress detailed FAQ answers into brief references. The Turbo Mode fallback prompt and the full pipeline Layer 7 prompt had similar issues.

Fix: Updated three prompts:

- `adapt_answer_for_fuzzy_match()`: “MUST reproduce the FULL and COMPLETE answer text.” Max tokens: 512 → 1500.
- `_turbo_pipeline()` LLM fallback: “MUST reproduce FULL and COMPLETE answer.”
- `response_generator()` Rule 8: “CRITICAL — COMPLETE ANSWERS: reproduce full FAQ answer text.”

Fix 4: Scanned PDF Vision API Fallback (`extraction_orchestrator.py`)

Problem: Scanned/image-only PDFs (no text layer) produced 0 chunks in Fast Mode because Vision API extraction was disabled.

Fix: Added automatic detection of scanned PDFs (PyMuPDF returns 0 text). When detected, Vision API is activated as a fallback regardless of Fast Mode setting.

Fix 5: FAISS/Chunk Mismatch Sync (`persistence_manager.py` — `load_into_indexer`)

Problem: After document deletion, FAISS could have more vectors than MongoDB chunks (e.g., 1253 vectors vs 991 chunks), causing “list index out of range” errors in Turbo Mode vector search.

Fix: Added comprehensive FAISS/chunk sync logic on startup:

1. Detects mismatch between FAISS vector count and MongoDB chunk count.
2. Matches vectors to chunks by `chunk_id` using FAISS metadata.
3. Re-embeds any unmatched chunks.
4. Rebuilds the FAISS index to match the synced state.
5. Added bounds checking in `DocumentIndexer.search()` as a safety net.

Fix 6: Admin Document Management (`app.py`, `seven_layer_rag.py`, `faq_exact_match.py`, `hybrid_search.py`)

New Feature: Admin Panel now includes a Document Management section with:

- View all uploaded documents with full metadata.

- Delete documents with confirmation dialog.
- Cascading deletion across all 5 stores (MongoDB, FAISS, in-memory indexer, FAQ engine, hybrid search).

---

17. Local Setup Guide

Prerequisites

- **Python** 3.10 or higher
- **MongoDB** Community Edition (running locally on port 27017)
- **OpenAI API Key** (for LLM calls)

Installation

```
# 1. Unzip the project
unzip banking-rag-app.zip -d banking-rag-app
cd banking-rag-app

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set environment variables
# Windows CMD:
set OPENAI_API_KEY=sk-your-key-here

# Windows PowerShell:
$env:OPENAI_API_KEY = "sk-your-key-here"

# macOS/Linux:
export OPENAI_API_KEY="sk-your-key-here"

# 4. Start MongoDB (if not running as a service)
mongod --dbpath /data/db

# 5. Run the application
streamlit run app.py
```

First-Time Setup

1. Open **http://localhost:8501** in your browser.
2. Login with `admin` / `admin88$`.
3. Upload your FAQ PDF documents via the sidebar file uploader.
4. Wait for extraction to complete (progress shown in sidebar).
5. Start querying in the Chat tab.

Clearing Old Data

If upgrading from a previous version, clear old FAQ pairs to force re-extraction:

```
mongosh banking_rag --eval "db.faq_pairs.drop()"
```

Or clear everything:

```
mongosh banking_rag --eval "db.dropDatabase()"
```

Also delete the FAISS index files:

```
rm -rf data/faiss_index/*
```

---

18. API and Model Dependencies

LLM Models Used

Model	Purpose	Where Used
`gpt-4.1-mini`	Primary LLM for all pipeline layers	Layers 2, 4, 5, 6, 7, Governance
`gpt-4.1-nano`	Lightweight model for FAQ fuzzy adaptation	FAQ Tier 2 only

Embedding Models

Model	Purpose	Dimension
`all-MiniLM-L6-v2`	Local sentence-transformers model for chunk/query embeddings	384

Python Dependencies

Package	Purpose
`streamlit`	Web application framework
`openai`	LLM API client
`pymongo`	MongoDB driver
`faiss-cpu`	Vector similarity search
`sentence-transformers`	Local embedding model
`PyMuPDF (fitz)`	PDF text extraction
`pdfplumber`	PDF table extraction
`numpy`	Numerical operations
`scikit-learn`	TF-IDF for BM25 hybrid search
`Pillow`	Image processing for vision extraction

End of Documentation